

GPU Architecture for Placement Interviews

This guide builds a practical mental model of modern GPU architecture. NVIDIA terminology is used where it is common in interviews; AMD equivalents are noted when useful. Exact sizes, policies, and limits vary by GPU generation, so distinguish architectural principles from device-specific numbers.

CPU vs GPU

1. Overview

A **CPU** is optimized to finish a small number of varied, latency-sensitive instruction streams quickly. A **GPU** is optimized to execute the same or similar work across thousands of data elements with high throughput.

- **Definition:** CPUs use a few powerful cores; GPUs use many simpler execution lanes grouped into parallel processors.
- **Why it matters:** Choosing the wrong processor can leave most hardware idle or add transfer and scheduling overhead that exceeds the useful work.
- **Real systems:** CPUs run operating systems, request handling, control logic, and serial code. GPUs accelerate graphics, machine learning, simulation, analytics, and media processing.
- **Why interviewers ask:** It reveals whether you understand latency versus throughput, parallelism, memory behavior, and heterogeneous programming.

2. Core Idea

Think of a CPU as a few expert chefs who can rapidly handle different custom orders. A GPU is a large kitchen line that can prepare thousands of similar items at once. The CPU minimizes the completion time of one task; the GPU maximizes completed work per unit time.

For vector addition $C[i] = A[i] + B[i]$:

1. The CPU may use a loop, several CPU threads, and SIMD instructions.
2. The GPU launches thousands of lightweight threads.
3. Each GPU thread handles one or a few indices.
4. Threads execute in groups, and memory requests are combined when addresses are adjacent.
5. The GPU wins only if the problem is large and regular enough to repay launch and data-transfer costs.

3. Important Subtopics

Latency versus throughput

- **Meaning:** Latency is time for one operation; throughput is operations completed per second.
- **Why it matters:** CPUs target low single-thread latency, while GPUs tolerate individual latency to sustain aggregate throughput.
- **Example:** A branch-heavy parser often favors a CPU; multiplying large matrices favors a GPU.
- **Interview angle:** A GPU instruction is not necessarily individually faster. Many operations run concurrently.

Core complexity

- **Meaning:** CPU cores spend substantial silicon on branch prediction, out-of-order execution, large caches, and speculative execution. GPU execution lanes devote more area to arithmetic throughput.
- **Why it matters:** This explains why “more GPU cores” does not mean each core resembles a CPU core.
- **Example:** One CPU core efficiently follows irregular pointer chains; many GPU lanes efficiently perform identical arithmetic.
- **Interview angle:** Compare architectural goals, not raw core counts.

Heterogeneous execution

- **Meaning:** The CPU, or **host**, launches GPU **kernels** on a device and often coordinates memory movement.
- **Why it matters:** Real applications divide work according to each processor’s strengths.
- **Example:** The CPU loads and validates an image; the GPU applies a convolution to every pixel.
- **Interview angle:** Include launch overhead, synchronization, and PCIe/interconnect transfers in performance reasoning.

Parallelism and control flow

- **Meaning:** GPUs work best when many threads follow similar instruction paths.
- **Why it matters:** Divergent branches and insufficient parallel work reduce utilization.
- **Example:** Dense image filtering is regular; recursive tree traversal is irregular.
- **Interview angle:** Explain why a parallel algorithm can still run poorly on a GPU.

4. Real-World Example

In an image-search backend, the CPU receives requests, decodes formats, performs authentication, and schedules batches. The GPU runs a neural network over a batch of images. Batching exposes enough parallel work to keep the GPU busy. For one tiny image, CPU execution may be faster because GPU launch and transfer overhead dominate.

5. Diagrams / Mental Models

CPU: few complex cores

```
+-----+ +-----+
| large  | | large  |
| cache +| | cache +|
| control| | control|
+-----+ +-----+
```

GPU: many throughput lanes

```
+---+---+---+---+---+---+
|ALU lanes in execution groups|
+---+---+---+---+---+---+
shared control and memory system
```

Host code -> allocate/copy -> launch kernel -> synchronize/copy -> host code

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why is a GPU faster for matrix multiplication?	It exposes massive regular data parallelism, reuses tiles, and performs many fused multiply-adds concurrently.	Parallelism, reuse, throughput.	"GPU clock is faster."
2. Is a GPU always faster than a CPU?	No. Small, serial, branch-heavy, or transfer-heavy work often favors a CPU.	Overheads and workload fit.	Treating speedup as automatic.
3. Why do CPUs have larger caches?	CPUs optimize unpredictable, latency-sensitive execution; GPUs rely more on concurrency to hide latency.	Latency-hiding distinction.	Saying GPUs have no cache.
4. What is host-device transfer overhead?	Time and bandwidth spent moving data between CPU and discrete GPU memory, often over PCIe.	End-to-end measurement.	Timing only the kernel.
5. What kind of workload suits a GPU?	Large, parallel, compute-dense work with regular control and memory access.	Several conditions, not only parallelism.	"Any loop."
6. Why are CPU and GPU core counts incomparable?	A CPU core is a powerful independent core; a listed GPU core is usually a much simpler arithmetic lane.	Architectural meaning of "core."	Comparing counts directly.
7. How does a GPU tolerate memory latency?	It schedules another ready warp/wavefront while one waits.	Hardware multithreading.	Claiming all accesses are fast.
8. What is kernel-launch overhead?	Fixed host/runtime/device work needed to enqueue and start a GPU function.	Small kernels can be overhead-bound.	Ignoring asynchronous launch.
9. Integrated versus discrete GPU?	Integrated GPUs commonly share system memory; discrete GPUs usually have dedicated high-bandwidth memory.	Transfer and bandwidth implications.	Assuming unified address means zero cost.
10. Latency or throughput: what does a GPU optimize?	Primarily throughput, though exact latency varies by operation and architecture.	Precise trade-off.	Saying latency is irrelevant.

7. Deep-Dive Questions

1. **Why can a theoretically parallel algorithm still lose on a GPU?** Too little work, excessive transfers, poor memory locality, divergence, synchronization, or low occupancy can outweigh parallel execution.
2. **How does Amdahl's law apply?** Speeding up only the GPU-friendly fraction limits total speedup:

$$\text{speedup} = 1 / ((1-p) + p/s).$$
3. **What changes with unified memory?** Address management becomes simpler, but page migration, coherence, bandwidth, and locality still matter; it does not erase the hierarchy.
4. **Can a CPU also perform data-parallel work?** Yes. CPU SIMD and multicore threading are highly effective, especially for moderate data sizes and low-latency tasks.
5. **Why batch GPU work?** Batching amortizes launch and transfer costs and exposes enough work to occupy the machine, at the possible cost of higher request latency.

8. Comparison Tables

Dimension	CPU	GPU
Primary goal	Low latency, generality	High throughput
Cores/execution lanes	Few, complex	Many, simpler
Control flow	Strong at irregular branches	Best with uniform paths
Scheduling	OS threads plus complex core logic	Lightweight hardware thread groups
Cache strategy	Large caches reduce latency	Caches plus massive latency hiding
Best workloads	Serial/control-heavy, small tasks	Large regular data-parallel tasks
Typical role	Host orchestration	Kernel acceleration

9. Common Mistakes

- Equating a GPU "core" with a CPU core.
- Claiming GPUs replace CPUs rather than complement them.
- Measuring kernel time while excluding transfers and synchronization.
- Assuming more threads always improve performance.
- Calling GPUs good only for graphics; they are general throughput processors.

10. Edge Cases / Special Cases

- Integrated GPUs may share physical memory with the CPU, changing transfer costs but not eliminating contention.
- A tiny compute-heavy kernel may still be slower because launch overhead is fixed.
- Dynamic parallelism lets some GPUs launch child kernels, but it has overhead and is not a substitute for good host scheduling.
- Modern CPUs have wide SIMD and many cores; modern GPUs also have caches and sophisticated control. The distinction is a spectrum.

11. How to Explain in Interview

"A CPU uses a few sophisticated cores to minimize latency for varied and irregular tasks. A GPU uses many simpler lanes to maximize throughput for large, regular, data-parallel work. GPUs hide latency by switching

among many ready thread groups, but launches, transfers, divergence, and memory access patterns determine whether acceleration is worthwhile.”

12. Quick Revision Notes

- CPU: latency-oriented; GPU: throughput-oriented.
- GPU sweet spot: large, regular, parallel, compute-dense work.
- Include data movement and launch cost in timing.
- More cores does not make CPU/GPU core counts comparable.
- Trap: “GPU is always faster.”

13. Practice Tasks

1. Implement vector addition on CPU and GPU; time transfer, kernel, and total execution separately.
2. Vary vector size and find the crossover point at which the GPU wins.
3. Classify parsing, sorting, image blur, graph traversal, and matrix multiply by likely processor fit.
4. Use Amdahl’s law to calculate total speedup when 90% of a program becomes 20× faster.

14. Final Cheat Sheet

Item	Remember
Core definition	CPU minimizes latency; GPU maximizes parallel throughput.
Why it matters	Workload shape determines processor choice.
Most asked	Why GPU for matrices? Is GPU always faster?
Key comparison	Few complex cores versus many simpler lanes.
One-line answer	“Use a GPU when enough regular parallel work amortizes movement and launch overhead.”

SIMD vs SMT

1. Overview

SIMD means **Single Instruction, Multiple Data**: one instruction explicitly operates on a vector of values.

SMT means **Single Instruction, Multiple Threads**: programmers describe independent scalar threads, while GPU hardware groups them and commonly executes one instruction across multiple lanes.

- **Why it matters**: It explains GPU execution, divergence, masking, and why the thread programming model differs from vector intrinsics.
- **Used in**: CPU vector extensions, GPU kernels, image processing, numerical code, and ML.
- **Interview value**: It tests whether you distinguish a programming/execution model from a machine instruction style.

2. Core Idea

SIMD is like one teacher giving the same arithmetic instruction to a row of students, with the row explicitly represented as a vector. SIMT is like writing instructions for each student independently; the school groups students who are currently at the same step.

Example: add 1 to eight values.

- SIMD: load a vector, issue one vector add, store a vector.
- SIMT: launch eight logical threads; thread `i` executes `a[i] += 1`.
- GPU hardware groups those threads into a warp/wavefront and executes their common instruction on lanes.
- If threads choose different branches, the hardware uses masks or schedules paths separately, reducing active-lane efficiency.

3. Important Subtopics

Vector width versus thread-group width

- **Meaning:** SIMD instructions have an architectural vector width; SIMT uses a warp/wavefront grouping chosen by the GPU architecture.
- **Why:** It affects utilization and portability.
- **Example:** AVX2 can process eight 32-bit floats; an NVIDIA warp contains 32 threads.
- **Interview angle:** Do not assume all GPUs use warp size 32; AMD commonly uses wavefront terminology and architecture-dependent widths.

Divergence and masks

- **Meaning:** Threads in a group may take different control paths. Lanes not participating in the current path are masked off.
- **Why:** Instructions may execute with only a fraction of lanes doing useful work.
- **Example:** Even thread IDs take branch A and odd IDs branch B.
- **Interview angle:** Divergence hurts primarily when it occurs within a warp/wavefront, not merely somewhere in a block.

Per-thread state

- **Meaning:** SIMT exposes thread IDs, registers, and logical control flow per thread.
- **Why:** It makes scalar-looking parallel code easy to express.
- **Example:** Each thread computes its array index from `blockIdx`, `blockDim`, and `threadIdx`.
- **Interview angle:** Threads are logically independent even when physically issued together.

Predication

- **Meaning:** A condition controls whether a lane commits an instruction rather than performing a full branch.
- **Why:** Short branches can be cheaper as predicated instructions.
- **Example:** `if (x > 0) y = x` can execute one masked assignment.
- **Interview angle:** Compilers and hardware may choose predication; not every `if` causes costly serialized paths.

4. Real-World Example

A browser applies a color transform to millions of pixels. On a CPU, vectorized code processes several pixels with each SIMD instruction. On a GPU, one logical thread handles one pixel, while SIMT hardware executes groups of pixels together. If pixels require many different code paths, lane utilization falls.

5. Diagrams / Mental Models

```

SIMD source view:      add vectorA, vectorB -> vectorC
                        [0 1 2 3] lanes

SIMT source view:      thread 0: c[0]=a[0]+b[0]
                        thread 1: c[1]=a[1]+b[1]
                        thread 2: c[2]=a[2]+b[2]
                        thread 3: c[3]=a[3]+b[3]

Hardware issue view:   one common instruction -> four active lanes
  
```

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is SIMD?	One instruction explicitly operates on multiple data lanes.	Vector execution.	Calling it multithreading.
2. What is SIMT?	Many logical scalar threads are grouped for common instruction issue.	Logical threads plus grouped execution.	Saying threads always run independently in hardware.
3. Main difference?	SIMD exposes vectors; SIMT exposes threads and lets hardware group them.	Programming-model distinction.	Claiming no relationship.
4. What is divergence?	Threads in one group follow different control paths, reducing active lanes or requiring separate path execution.	Scope and consequence.	Saying any branch is divergence.
5. Does divergence produce wrong results?	No, hardware masks/schedules lanes to preserve per-thread semantics; it usually affects performance.	Correctness versus speed.	Saying both branches run and overwrite values.
6. Is SIMT just SIMD?	Hardware may be SIMD-like, but SIMT adds per-thread state and a thread-centric execution model.	Nuance.	Treating terms as exact synonyms.

Question	Clear answer	Interviewer expects	Common mistake
7. How can divergence be reduced?	Group similar work, restructure algorithms, reduce branch imbalance, or use predication when appropriate.	Data organization.	Removing all conditions blindly.
8. What happens when half a warp takes each branch?	Paths may be executed under different masks, so roughly half the lanes are useful on each path.	Mask mental model.	Saying two warps are automatically created.
9. Are loops divergent?	They are when threads in a group execute different iteration counts.	Control-flow scope.	Focusing only on <code>if</code> .
10. Does CPU SIMD have divergence?	Traditional SIMD uses explicit masks or branches; modern vector ISAs support masked/predicated operations.	Masking comparison.	Saying SIMD cannot express conditions.

7. Deep-Dive Questions

- 1. What is independent thread scheduling?** Newer GPU architectures can maintain finer per-thread execution state, improving synchronization flexibility, but lanes still share execution resources and divergence still costs throughput.
- 2. Why is divergence sometimes harmless?** If all threads choose the same path, the branch is uniform; very short branches may also compile to predication.
- 3. How do reconvergence mechanisms work conceptually?** Hardware tracks control-flow paths and active masks, then brings threads back together at a reconvergence point.
- 4. How is a masked vector instruction related to SIMT?** Both apply an operation only to active lanes, but the programmer-visible state and scheduling model differ.
- 5. Can divergence cross warp boundaries?** Different warps can take different paths without intra-warp lane wastage; they are separately scheduled.

8. Comparison Tables

Dimension	SIMD	SIMT
Programmer sees	Vector values/instructions	Scalar logical threads
State	Per vector plus lane masks	Per-thread registers and control state
Typical platform	CPU vector units, vector processors	GPUs
Width handling	ISA/compiler vector width	Hardware thread-group width
Conditional work	Masks/predication/vector branches	Thread divergence and active masks
Example	AVX vector add	CUDA/OpenCL kernel threads

9. Common Mistakes

- Saying SIMT means every thread has a physically independent instruction unit.
- Assuming divergence occurs across the whole grid.
- Treating every branch as expensive divergence.
- Hard-coding a warp size without checking the target architecture.
- Ignoring divergent loop trip counts.

10. Edge Cases / Special Cases

- A branch with a uniform condition does not create intra-group divergence.
- Predication can execute both instruction sequences while suppressing writes, which is efficient only for short paths.
- Some vector architectures use scalable vector lengths; SIMD is not always a fixed source-level width.
- Independent thread scheduling changes some old lockstep assumptions, especially for warp-synchronous code; explicit synchronization is safer.

11. How to Explain in Interview

"SIMD exposes a vector instruction that processes multiple data lanes. SIMT exposes many scalar threads, and GPU hardware schedules them in warps or wavefronts that usually issue a common instruction across lanes. SIMT is easier to program per element, but different paths within a group cause divergence and lower lane utilization."

12. Quick Revision Notes

- SIMD: vector is explicit; SIMT: logical threads are explicit.
- SIMT provides per-thread IDs, registers, and control state.
- Divergence matters inside a warp/wavefront.
- Uniform branches are not divergent.
- Trap: SIMT and SIMD are related, not identical.

13. Practice Tasks

1. Write a scalar array-add loop and inspect whether the CPU compiler vectorizes it.
2. Write a GPU kernel with even/odd branches; reason about active masks.
3. Reorder data so each warp handles similar cases and compare performance.
4. Draw active-lane masks for a four-lane group through nested branches.

14. Final Cheat Sheet

Item	Remember
Core definition	SIMD applies one vector instruction; SIMT groups scalar threads.
Why it matters	It explains GPU control flow and utilization.
Most asked	SIMD vs SIMT; what is divergence?
Key comparison	Explicit vectors versus explicit threads.
One-line answer	"SIMT gives a thread abstraction over grouped lane execution."

Streaming Multiprocessors (SMs)

1. Overview

A **Streaming Multiprocessor (SM)** is an NVIDIA GPU's main programmable execution cluster. AMD uses related concepts such as Compute Units (CUs) or Work Group Processors, depending on architecture.

- **Definition:** An SM contains execution units, warp schedulers, registers, shared memory, caches, and control hardware needed to run resident thread blocks.
- **Why it matters:** Kernel performance depends on how blocks and warps occupy SM resources.
- **Used in:** Every general-purpose GPU kernel and graphics shader.
- **Interview value:** SMs connect the programming hierarchy to actual hardware scheduling and resource limits.

2. Core Idea

Imagine a GPU as a factory and each SM as one workshop. A thread block is assigned to one workshop and stays there until completion. The workshop keeps several warps resident. When one warp waits for memory, a scheduler selects another ready warp.

Step by step:

1. A kernel grid contains many blocks.
2. Hardware dispatches blocks to available SMs.
3. Each block reserves registers, shared memory, thread slots, and block slots on its SM.
4. Warp schedulers issue instructions from ready resident warps to execution units.
5. Completed blocks release resources, allowing new blocks to enter.

3. Important Subtopics

Residency and occupancy

- **Meaning:** Resident warps/blocks are loaded on an SM. Occupancy is resident active warps relative to the hardware maximum.
- **Why:** More resident warps can hide latency, but maximum occupancy is not automatically maximum performance.
- **Example:** High register use per thread may allow fewer blocks to reside.
- **Interview angle:** Discuss limiting resources and useful occupancy, not a blind 100% target.

Warp schedulers and execution units

- **Meaning:** Schedulers choose ready warps; functional units execute integer, floating-point, load/store, special-function, or matrix instructions.
- **Why:** Instruction mix and dependencies affect which units are busy.
- **Example:** A compute loop may saturate FP units while load/store units are underused.
- **Interview angle:** An SM does not execute one whole block instruction at once; it schedules warps.

Resource partitioning

- **Meaning:** Registers and shared memory are finite pools divided among resident blocks.
- **Why:** Per-block and per-thread use determines concurrency.
- **Example:** Reducing registers from 80 to 64 per thread might permit another resident block, depending on allocation granularity.
- **Interview angle:** Explain resource trade-offs and possible register spilling.

Block scheduling

- **Meaning:** A block runs entirely on one SM; blocks may execute in any order.
- **Why:** Blocks must be independently schedulable unless special cooperative mechanisms are used.
- **Example:** A grid cannot normally use a simple barrier across all blocks inside a kernel.
- **Interview angle:** Block independence enables scaling across different SM counts.

4. Real-World Example

For a tiled matrix multiplication, each block computes one output tile. When assigned to an SM, it loads input tiles into that SM's shared memory, synchronizes its threads, performs multiply-adds, and writes results. Multiple blocks may reside on one SM if registers, shared memory, and thread limits permit.

5. Diagrams / Mental Models

```
GPU
├── SM 0: [warp schedulers] [ALUs/Tensor] [register file] [shared/L1]
│       block 3 -> warps 0..7      block 9 -> warps 0..7
├── SM 1: block 1, block 8, ...
└── SM N: block 2, block 6, ...

Waiting warp ----> scheduler chooses another ready resident warp
```

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is an SM?	A GPU execution cluster that schedules warps and contains execution and local storage resources.	Components and role.	Calling it one GPU core.
2. Where does a block run?	One block is resident on one SM for its lifetime.	Scheduling unit.	Splitting one block across SMs.
3. Can one SM run multiple blocks?	Yes, if block/thread/register/shared-memory and architectural limits allow.	Resource-based residency.	Assuming exactly one block.

Question	Clear answer	Interviewer expects	Common mistake
4. Can a block move between SMs?	Normally no; it remains on its assigned SM until completion.	Lifetime placement.	Treating it like an OS thread.
5. What is occupancy?	Resident active warps divided by the SM's supported maximum.	Formal ratio and purpose.	Equating it with ALU utilization.
6. Why keep many warps resident?	To hide stalls by issuing another ready warp.	Latency hiding.	Saying it makes memory faster.
7. What limits occupancy?	Threads/block, blocks/SM, warps/SM, registers, shared memory, and allocation granularity.	Multiple constraints.	Naming only block size.
8. Is 100% occupancy required?	No. Enough warps to hide latency may suffice; instruction-level parallelism, cache behavior, and resource use also matter.	Nuance.	Maximizing occupancy at all costs.
9. What happens to shared memory between blocks?	It is allocated per resident block and released when the block completes.	Scope/lifetime.	Assuming persistence across blocks.
10. How are blocks distributed?	Dynamically to SMs as resources become available; programming should not depend on ordinary block order.	Scheduling independence.	Assuming increasing block index order.

7. Deep-Dive Questions

1. **How do register limits affect residency?** Required registers are allocated per warp/block with hardware granularity; the finite SM register file may reduce resident blocks or cause compiler spilling.
2. **Why can lower occupancy be faster?** More registers per thread may reduce loads and expose instruction-level parallelism; larger shared-memory tiles may improve reuse.
3. **What is a scoreboard stall?** The scheduler cannot issue an instruction because an input result, often from memory or an earlier instruction, is not ready.
4. **Can blocks communicate through shared memory?** Only threads within the same block, because different blocks can be on different SMs and run at different times.
5. **What are cooperative launches?** Special APIs can guarantee conditions enabling grid-wide coordination, but they constrain residency and are not the default execution model.

8. Comparison Tables

Dimension	SM / Compute Unit	CPU core
Scheduling unit	Warps/wavefronts	Hardware/OS threads
Resident contexts	Many lightweight warps	Usually few hardware threads
Local storage	Large register file, shared/local memory	Registers and cache hierarchy

Dimension	SM / Compute Unit	CPU core
Optimization	Throughput, latency hiding	Single-thread latency
Work assignment	Thread blocks/work-groups	Processes/threads/tasks

9. Common Mistakes

- Using SM, CUDA core, and warp as synonyms.
- Assuming blocks execute in index order.
- Assuming a block can span several SMs.
- Chasing occupancy while causing spills or reducing tile reuse.
- Believing resident means currently executing every cycle.

10. Edge Cases / Special Cases

- Resource allocation often has granularity, so small source changes can cause stepwise occupancy changes.
- Shared memory and L1 may share configurable physical capacity on some architectures.
- Persistent kernels deliberately keep blocks resident and pull work from queues; they require careful load balancing.
- Hardware generation determines scheduler counts and issue rules; avoid memorizing one generation as universal.

11. How to Explain in Interview

“An SM is the GPU cluster that actually hosts blocks and schedules their warps onto execution units. A block reserves a share of the SM’s registers, shared memory, and thread capacity and stays on that SM. Multiple resident warps let the scheduler hide stalls, while resource use determines occupancy.”

12. Quick Revision Notes

- Block placement: one SM for the block’s lifetime.
- SM owns schedulers, execution units, register file, shared memory/L1.
- Residency is constrained by several finite resources.
- Occupancy helps latency hiding but is not the same as utilization.
- Blocks normally must be order-independent.

13. Practice Tasks

1. Use a GPU occupancy calculator for several block sizes and register counts.
2. Modify shared-memory tile size and record both occupancy and kernel time.
3. Draw how ten blocks could be scheduled on three SMs without assuming order.
4. Inspect profiler stall reasons for a memory-latency-bound kernel.

14. Final Cheat Sheet

Item	Remember
Core definition	SM = block-hosting, warp-scheduling execution cluster.
Why it matters	It determines residency, latency hiding, and resource contention.
Most asked	Can a block span SMs? What limits occupancy?
Key comparison	SM schedules many warps; CPU core runs a few heavyweight contexts.
One-line answer	"Blocks live on SMs; SM resources decide how many can live there together."

Warps / Wavefronts

1. Overview

A **warp** is NVIDIA's basic group of threads scheduled for instruction issue; a **wavefront** is the closely related AMD term. Threads in the group usually execute the same instruction on different data lanes.

- **Definition:** A fixed hardware grouping of neighboring logical threads.
- **Why it matters:** Divergence, memory coalescing, occupancy, and scheduling are understood at this level.
- **Used in:** All GPU kernels and shaders.
- **Interview value:** It tests whether you know how individually written threads become hardware work.

2. Core Idea

Imagine 32 travelers moving as a tour group. They can carry different luggage and calculate different values, but the guide issues a common instruction. If half visit room A and half room B, the guide may lead each subgroup in turn while the other waits.

For a 256-thread NVIDIA block:

1. Threads are partitioned into eight 32-thread warps.
2. Each resident warp has per-thread state and an active mask.
3. A scheduler selects a ready warp.
4. The issued instruction operates on active lanes.
5. Memory operations are combined into transactions according to addresses and access size.

3. Important Subtopics

Formation and size

- **Meaning:** Consecutive thread IDs are grouped according to a linearized thread index.
- **Why:** Block dimensions affect which data elements share a warp.
- **Example:** A 2D block is linearized with `x` varying fastest in CUDA.
- **Interview angle:** Warp size is architecture-specific; query or use platform constants.

Divergence

- **Meaning:** Lanes in one warp/wavefront require different instruction paths.
- **Why:** Some lanes become inactive during each path.
- **Example:** `if (threadIdx.x < 16)` splits one 32-thread warp evenly.
- **Interview angle:** Divergence is a utilization cost, not generally a correctness issue.

Coalescing

- **Meaning:** Lane memory accesses are serviced with as few memory transactions as practical.
- **Why:** Efficient coalescing raises effective bandwidth.
- **Example:** Lane `i` reading `a[base+i]` is favorable; large-stride or scattered accesses may require more transactions.
- **Interview angle:** Coalescing depends on addresses, alignment, access size, and architecture—not just “consecutive threads.”

Warp-level primitives

- **Meaning:** Shuffle, ballot, vote, and warp-level reduce operations exchange data or decisions within a group.
- **Why:** They can avoid shared-memory round trips.
- **Example:** A shuffle-down sequence performs a warp reduction.
- **Interview angle:** Use correct active masks and synchronization semantics.

4. Real-World Example

In database filtering on a GPU, each lane examines one row and produces a predicate. A ballot operation packs 32 Boolean results into a bit mask, and a prefix operation computes compact output positions. If adjacent lanes read adjacent columns, loads coalesce; highly variable per-row processing causes divergence.

5. Diagrams / Mental Models

Block of 96 threads (warp size 32)

Warp 0: threads 0..31 active mask: 111111...111

Warp 1: threads 32..63 active mask: 111100...001

Warp 2: threads 64..95 waiting on memory

Cycle idea: scheduler picks a READY warp, not necessarily warp 0 then 1 then 2.

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is a warp?	A hardware scheduling/execution group of neighboring GPU threads.	Group role.	Calling it a block.
2. Typical NVIDIA warp size?	Commonly 32 threads, but code should use platform-provided values where possible.	32 plus portability.	Applying 32 to all vendors.

Question	Clear answer	Interviewer expects	Common mistake
3. What is a wavefront?	AMD's related thread-execution group, with width depending on architecture/mode.	Vendor distinction.	Saying it is identical in every detail.
4. How many warps in 256 threads?	Eight when warp size is 32: ceiling of <code>threads/warpSize</code> .	Ceiling calculation.	Integer truncation for partial warp.
5. What is a partial warp?	The final warp has fewer valid threads when block size is not a multiple of warp size.	Inactive lanes.	Assuming no warp is allocated.
6. What is warp divergence?	Lanes follow different paths, causing masked or separately scheduled execution.	Active mask.	Confusing with different warps choosing paths.
7. What is coalescing?	Combining lane accesses into efficient memory transactions.	Address pattern.	Calling it caching.
8. Do all resident warps execute simultaneously?	No. Schedulers issue from ready warps over time to available units.	Residency versus issue.	Equating resident with issued.
9. Why are warp-level operations fast?	They exchange lane data through specialized cross-lane hardware without ordinary shared/global-memory traffic.	Shuffle/ballot benefit.	Assuming they work across blocks.
10. How does a memory stall get hidden?	The scheduler issues another ready resident warp.	Zero/low-cost warp selection concept.	Claiming the stalled warp moves to another SM.

7. Deep-Dive Questions

- How are multidimensional threads placed into warps?** Thread indices are linearized, conventionally with the x dimension fastest, then consecutive linear IDs form groups.
- Why can AoS hurt coalescing?** A structure stride may make lanes touch separated fields/segments; SoA often makes a given field contiguous across lanes.
- What is warp execution efficiency?** A profiler metric approximating active lanes per issued warp instruction; divergence and partial warps can reduce it.
- When is warp-synchronous programming unsafe?** When it assumes implicit lockstep or memory visibility not guaranteed by the architecture; use documented warp synchronization and masks.
- Why can different warps diverge without a divergence penalty?** Each warp has its own instruction state, so the scheduler can issue their distinct paths separately; workload imbalance may still matter.

8. Comparison Tables

Dimension	Warp (NVIDIA)	Wavefront (AMD)	Thread block/work-group
Level	Hardware execution group	Hardware execution group	Programmer-visible cooperation group
Typical width	Commonly 32	Architecture/mode-dependent	User selected, usually many groups
Shared memory scope	Via containing block	Via containing work-group	Entire block/work-group
Scheduling	Issued by SM schedulers	Issued by CU schedulers	Assigned as a unit to one SM/CU
Divergence scope	Within warp	Within wavefront	Each constituent group separately

9. Common Mistakes

- Treating a warp as a user-created synchronization object.
- Assuming different warps execute in lockstep.
- Ignoring the partially filled last warp.
- Assuming adjacent threads guarantee one memory transaction in every case.
- Using warp intrinsics without the correct active-lane mask.

10. Edge Cases / Special Cases

- A block smaller than one warp still consumes a warp with inactive lanes.
- Independent thread scheduling means old implicit warp-synchronous patterns may race.
- Misaligned contiguous accesses may need extra memory transactions.
- Divergence can be nested, and loop exit conditions can create changing active masks.
- Wave sizes and execution modes vary; avoid embedding vendor-specific constants in portable algorithms.

11. How to Explain in Interview

"A warp or wavefront is the hardware group in which neighboring GPU threads are scheduled. Each lane has its own data and state, but lanes commonly share instruction issue. Branch divergence reduces active lanes, while contiguous lane addresses help coalesce memory transactions. Multiple resident groups let the SM hide latency."

12. Quick Revision Notes

- NVIDIA warp: commonly 32 threads.
- Warps are formed from consecutive linear thread IDs.
- Number of warps is `ceil(blockThreads / warpSize)`.
- Divergence and coalescing are warp-level concerns.
- Resident is not the same as currently issued.

13. Practice Tasks

1. Compute warp counts and inactive lanes for block sizes 32, 48, 128, 250, and 256.
2. Draw masks for a warp executing `if (lane < 10)`, then `if (lane % 4 == 0)`.
3. Compare contiguous, stride-2, and random global-memory accesses in a profiler.
4. Implement a warp-level sum using shuffle operations and verify partial-warp handling.

14. Final Cheat Sheet

Item	Remember
Core definition	Warp/wavefront = hardware thread execution group.
Why it matters	It determines divergence, coalescing, and issue behavior.
Most asked	Warp size? Divergence? Coalescing?
Key comparison	Warp is hardware-level; block is cooperation/scheduling-level.
One-line answer	"Threads are written independently but issued in warp-sized groups."

Threads, Blocks, Grids

1. Overview

GPU programs express parallel work through a hierarchy: a **thread** performs one logical task, a **block** groups threads that can cooperate, and a **grid** contains all blocks launched for one kernel.

- **Definition:** Thread = smallest logical worker; block/work-group = cooperation and placement unit; grid/NDRange = complete kernel launch.
- **Why it matters:** Indexing, synchronization, memory scope, scalability, and performance all follow this hierarchy.
- **Used in:** CUDA kernels, HIP, OpenCL, graphics compute shaders, and accelerator frameworks.
- **Interview value:** Interviewers frequently ask for indexing formulas, synchronization scope, and block-size trade-offs.

2. Core Idea

Think of processing a large book. One thread handles one line, one block handles a page and shares a small worktable, and the grid covers the entire book. Pages can be processed in any order, so no page should require another unfinished page unless the algorithm uses a separate kernel phase.

For **N** elements with **B** threads per block:

1. Launch `ceil(N/B)` blocks.
2. Thread computes `i = blockIdx.x * blockDim.x + threadIdx.x`.
3. It checks `if (i < N)` because the last block may be partial.
4. Threads in a block may exchange data through shared memory and synchronize.
5. The kernel completes after every block completes; the next dependent kernel provides a grid-wide phase boundary.

3. Important Subtopics

Indexing and dimensionality

- **Meaning:** Threads, blocks, and grids can be 1D, 2D, or 3D for convenient data mapping.
- **Why:** Correct indexing prevents missed elements and out-of-bounds access.
- **Example:** Pixel (x, y) uses block and thread coordinates in two dimensions.
- **Interview angle:** Derive global and flattened indices, including bounds checks.

Cooperation and synchronization

- **Meaning:** Threads in a block can synchronize at a block barrier and share block-local memory.
- **Why:** It enables tiled algorithms and reductions.
- **Example:** Load a matrix tile, call a block barrier, then consume the tile.
- **Interview angle:** Every participating thread must reach a barrier consistently; a normal barrier is not grid-wide.

Block independence

- **Meaning:** Ordinary blocks may run in any order, concurrently or sequentially.
- **Why:** The same kernel scales across GPUs with different numbers of SMs.
- **Example:** Each image tile writes a disjoint output region.
- **Interview angle:** Avoid inter-block assumptions and deadlocking global spin barriers.

Block-size choice

- **Meaning:** Threads per block affect warp utilization, resource allocation, and available parallelism.
- **Why:** A good size provides enough warps without exceeding register/shared-memory limits.
- **Example:** 128 or 256 threads is a common starting point, not a universal optimum.
- **Interview angle:** Discuss warp multiples, occupancy, memory pattern, and measurement.

4. Real-World Example

For image blur, the grid covers the image, each 2D block covers an output tile, and each thread computes a pixel. The block cooperatively loads a tile plus halo pixels into shared memory. After a barrier, neighboring input values are reused from shared memory rather than repeatedly fetched from global memory.

5. Diagrams / Mental Models

```
Grid (one kernel launch)
├─ Block (0,0): thread (0,0) ... thread (15,15)
├─ Block (1,0): thread (0,0) ... thread (15,15)
├─ Block (0,1): ...
└─ Block (1,1): ...

globalX = blockIdx.x * blockDim.x + threadIdx.x
globalY = blockIdx.y * blockDim.y + threadIdx.y
```

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is a GPU thread?	A lightweight logical execution context that runs a kernel instance with its own indices and registers.	Logical, not OS thread.	Equating it to a CPU thread.
2. What is a block?	A group of threads assigned to one SM that can share memory and use block synchronization.	Cooperation and placement.	Saying it can span SMs.
3. What is a grid?	All blocks created by one kernel launch.	Launch-level scope.	Calling one block a grid.
4. Global 1D index?	<code>blockIdx.x * blockDim.x + threadIdx.x</code> .	Formula and bounds check.	Omitting block offset.
5. Why use a bounds check?	Grid size is commonly rounded up, so extra threads in the last block must not access invalid elements.	Ceiling launch.	Launching too few blocks.
6. Can blocks synchronize normally?	Not with a standard block barrier; use separate kernels, cooperative mechanisms, or other carefully designed primitives.	Scope and alternatives.	Using <code>__syncthreads()</code> grid-wide.
7. Why must blocks be independent?	Scheduling order is unspecified and hardware-dependent, enabling scalable execution.	Portability/scalability.	Assuming block index order.
8. How choose block size?	Start with a warp multiple such as 128/256, then account for registers, shared memory, access patterns, and profile.	Trade-offs and measurement.	"Always use 1024."
9. What does a block barrier guarantee?	Participating block threads wait, and documented memory effects make earlier shared/global writes visible within its scope.	Execution plus memory ordering.	Treating it as only a delay.
10. Can a thread belong to two blocks?	No. Each launched thread belongs to exactly one block in that grid.	Hierarchy clarity.	Confusing reuse across kernels.

7. Deep-Dive Questions

- What is grid-stride looping?** Each thread handles `i`, then `i += blockDim.x * gridDim.x`, allowing a fixed grid to cover arbitrary `N` and reuse threads.
- Why can a conditional barrier deadlock?** If some required block threads skip the barrier while others wait, the waiting threads may never be released.

3. **How is a multidimensional thread ID flattened?** In CUDA-style ordering, `linear = x + blockDim.x * (y + blockDim.y * z)`.
4. **Why split an algorithm into multiple kernels?** A kernel boundary supplies a natural global phase separation and permits all blocks from phase one to finish before phase two.
5. **What are thread clusters/cooperative groups?** Architecture/API-specific mechanisms expose cooperation beyond a simple block or provide finer group abstractions; they require explicit support and constraints.

8. Comparison Tables

Property	Thread	Block / work-group	Grid / NDRange
Represents	One logical worker	Cooperating workers	Entire kernel launch
Private state	Registers/local state	—	—
Shared fast storage	Accesses block shared memory	Own allocation	No ordinary grid-shared scratchpad
Barrier scope	Participates	Standard barrier covers block	Usually kernel boundary/special API
Hardware mapping	Lane in warp/wavefront	One SM/CU at a time	Distributed across device

9. Common Mistakes

- Forgetting the last-block bounds check.
- Using floor division to calculate grid size.
- Treating block index order as execution order.
- Calling a block barrier from only part of a block.
- Choosing dimensions convenient for code but poor for coalescing.

10. Edge Cases / Special Cases

- Zero-length input should usually avoid an invalid zero-block launch or use an API-valid no-op path.
- A partial last warp exists when block threads are not a warp-size multiple.
- Maximum dimensions and total threads per block are device-specific.
- Cooperative grid synchronization is valid only for launches satisfying residency and API requirements.
- Threads can process multiple elements; one thread per element is a mapping pattern, not a rule.

11. How to Explain in Interview

"A kernel launch creates a grid of blocks, and each block contains threads. A thread computes one logical portion using its indices. Threads within a block can share fast memory and synchronize because the block resides on one SM. Blocks normally cannot assume order or use a regular global barrier, which makes the grid portable across different GPU sizes."

12. Quick Revision Notes

- Thread → block → grid.
- Global index: block offset plus local thread index.
- Round up block count, then guard $i < N$.
- Block: synchronization and shared-memory scope.
- Grid-wide phases: commonly separate kernels.

13. Practice Tasks

1. Write 1D vector-add and 2D image-indexing kernels with bounds checks.
2. Flatten and unflatten 3D thread indices.
3. Implement a grid-stride loop and test sizes smaller and larger than the grid.
4. Explain why a software global barrier based on spinning block counters can deadlock.

14. Final Cheat Sheet

Item	Remember
Core definition	Threads do work, blocks cooperate, grids launch all work.
Why it matters	Defines indexing, memory scope, and synchronization.
Most asked	Global index? Can blocks synchronize?
Key comparison	Block is local cooperation; grid is global coverage.
One-line answer	"Map data to threads, cooperation to blocks, and the full problem to a grid."

GPU Cores vs Tensor Cores

1. Overview

General GPU arithmetic lanes—often marketed by NVIDIA as **CUDA cores**—execute scalar/vector-like arithmetic and logic instructions. **Tensor Cores** are specialized matrix-multiply-accumulate units designed to process small matrix tiles at very high throughput.

- **Definition:** General cores handle broad instruction types; Tensor Cores accelerate structured matrix operations, often with mixed precision.
- **Why it matters:** AI and dense linear algebra performance depends on using the right unit and supported data layout/precision.
- **Used in:** Shaders, simulations, ML training/inference, matrix multiplication, and scientific computing.
- **Interview value:** It exposes the difference between general programmability, specialization, precision, and advertised peak FLOPS.

2. Core Idea

A general arithmetic lane is like a calculator handling individual operations. A Tensor Core is like a matrix-calculation machine: feed it correctly shaped tiles and it performs many multiply-accumulates together.

For $D = A \times B + C$:

1. Threads cooperatively provide fragments/tiles of **A**, **B**, and **C**.
2. A matrix instruction maps the operation to Tensor Core hardware.
3. Multiplication may use lower precision such as FP16, BF16, TF32, FP8, or integer formats, depending on hardware.
4. Accumulation may use a wider type such as FP32.
5. Unsupported shape, layout, alignment, or code may fall back to ordinary instructions or require padding.

3. Important Subtopics

General-purpose arithmetic lanes

- **Meaning:** Execute floating-point, integer, and related instructions issued for threads.
- **Why:** Most kernel code, address calculation, control logic, and unsupported math uses them.
- **Example:** Vector addition and pointer arithmetic.
- **Interview angle:** "CUDA core" is vendor/marketing terminology, not an independent CPU-like core.

Matrix multiply-accumulate

- **Meaning:** Tensor units compute a tile-level $A \times B + C$ operation.
- **Why:** Matrix multiplication dominates many neural networks.
- **Example:** A convolution lowered or mapped to matrix multiplication.
- **Interview angle:** Tensor Cores do not accelerate arbitrary code.

Mixed precision

- **Meaning:** Input multiplication and accumulation may use different precisions.
- **Why:** Lower precision raises throughput and reduces bandwidth, while wider accumulation protects accuracy.
- **Example:** FP16 multiply with FP32 accumulation.
- **Interview angle:** Discuss numerical range, rounding, scaling, and model tolerance.

Utilization requirements

- **Meaning:** Shapes, strides, alignment, batching, and library selection determine whether tensor units are used efficiently.
- **Why:** Peak tensor throughput is irrelevant if tiles are tiny or data preparation dominates.
- **Example:** A vendor BLAS/DNN library selects a tuned Tensor Core kernel.
- **Interview angle:** Prefer optimized libraries before handwritten matrix kernels.

4. Real-World Example

During transformer inference, large linear layers map naturally to matrix multiplication and use Tensor Cores. The surrounding activation functions, indexing, normalization, and control logic use general GPU units. Small batch sizes or awkward dimensions may underutilize tensor units, so inference engines fuse operations and batch requests where latency constraints allow.

5. Diagrams / Mental Models

Kernel instruction mix

- |— addresses, branches, elementwise math -> general GPU units
- |— tile $D = A \times B + C$ -> Tensor Cores

FP16/BF16/TF32 inputs -> many parallel products -> often FP32 accumulation

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is a CUDA core?	A general arithmetic execution lane in an NVIDIA SM, not a CPU-style independent core.	Lane-level meaning.	Comparing one-to-one with CPU cores.
2. What is a Tensor Core?	Specialized hardware for tile matrix multiply-accumulate at high throughput.	MMA operation.	Calling it a complete AI processor.
3. Why are Tensor Cores faster?	Specialization performs many low-/mixed-precision multiply-accumulates per instruction and unit.	Structured work and precision.	"Higher clock speed."
4. Do Tensor Cores replace CUDA cores?	No. General units handle control, addressing, elementwise and unsupported operations.	Complementary roles.	Saying all ML runs only on Tensor Cores.
5. Can any kernel use Tensor Cores?	Only code mapped to supported matrix operations, types, layouts, and shapes.	Constraints.	Assuming compiler maps arbitrary loops.
6. What is mixed precision?	Lower-precision inputs/products combined with typically wider accumulation or selected output precision.	Performance/accuracy trade-off.	Assuming output is automatically exact.
7. What is TF32 conceptually?	A format/path designed to accelerate FP32-oriented matrix work with reduced mantissa precision while commonly accumulating in FP32.	Precision nuance.	Treating it as full IEEE FP32 multiplication.
8. How do programmers access Tensor Cores?	Usually through tuned BLAS/DNN/framework libraries, or lower-level matrix APIs/instructions.	Libraries first.	Handwriting everything.

Question	Clear answer	Interviewer expects	Common mistake
9. What can prevent utilization?	Small/awkward shapes, unsupported type/layout, poor alignment, data movement, or non-matrix workload.	Practical conditions.	Quoting only peak FLOPS.
10. Are Tensor Core results identical to FP32 scalar math?	Not necessarily; input precision, multiplication semantics, accumulation order, and rounding can differ.	Numerical awareness.	Expecting bitwise identity.

7. Deep-Dive Questions

1. **Why does accumulation precision matter?** A dot product adds many products; wider accumulation reduces rounding error and preserves small contributions.
2. **How does arithmetic intensity affect tensor performance?** Matrix tiles must be reused enough that memory bandwidth does not starve the high-throughput units.
3. **Why do libraries sometimes pad dimensions?** Supported tile sizes and alignment can make padded computation faster than an irregular edge kernel despite extra arithmetic.
4. **What is sparsity acceleration?** Some tensor hardware can skip work for supported structured sparsity patterns, but data must satisfy precise layout constraints.
5. **Why might a Tensor Core kernel be slower for a small matrix?** Setup, tile underfill, launch overhead, and insufficient parallelism can dominate.

8. Comparison Tables

Dimension	General GPU cores/lanes	Tensor Cores
Work	Scalar/thread arithmetic and logic	Matrix tile multiply-accumulate
Flexibility	Broad	Specialized
Precision	Many scalar types/instructions	Supported matrix types, often mixed precision
Best use	Elementwise, control, general kernels	Dense/supported structured linear algebra
Programming	Ordinary kernel instructions	Libraries or matrix APIs/instructions
Peak metric	General FP/INT throughput	Tensor/matrix throughput

9. Common Mistakes

- Treating tensor throughput numbers as achievable for every kernel.
- Ignoring numerical accuracy when reducing precision.
- Assuming a framework operation necessarily uses Tensor Cores.
- Comparing CUDA-core and Tensor-Core counts as equivalent units.
- Writing custom matrix code before checking tuned libraries.

10. Edge Cases / Special Cases

- Supported types and tile shapes change across hardware generations.
- Boundary tiles may use masks, padding, or fallback paths.
- Deterministic results can conflict with the fastest accumulation algorithms.
- Quantized integer or FP8 workloads need scale management to preserve range.
- Specialized sparse acceleration generally requires structured, not arbitrary, sparsity.

11. How to Explain in Interview

“General GPU cores execute ordinary per-thread arithmetic and control-related instructions. Tensor Cores are specialized units for small matrix multiply-accumulate operations and achieve much higher throughput for supported shapes and precisions. Real ML kernels use both, and performance depends on mapping work, feeding data, and accepting appropriate numerical precision.”

12. Quick Revision Notes

- General lanes: broad instruction support.
- Tensor Cores: tile matrix multiply-accumulate.
- Specialized means faster only for matching work.
- Mixed precision improves speed/bandwidth but changes numerical behavior.
- Tuned libraries are the normal entry point.

13. Practice Tasks

1. Benchmark the same GEMM with FP32 and a supported mixed-precision mode.
2. Test aligned and awkward matrix dimensions and inspect tensor-unit utilization.
3. Compare numerical error against a higher-precision reference.
4. Use a profiler to separate general ALU, Tensor Core, and memory bottlenecks.

14. Final Cheat Sheet

Item	Remember
Core definition	General lanes run broad operations; Tensor Cores run matrix MMA.
Why it matters	Matching dense linear algebra to specialized units yields large speedups.
Most asked	What is a Tensor Core? What is mixed precision?
Key comparison	Flexible scalar/thread execution versus specialized tile execution.
One-line answer	“Tensor Cores trade generality and often precision for exceptional matrix throughput.”

Registers

1. Overview

Registers are the fastest programmer-relevant storage used for a thread’s active values. GPU compilers normally assign scalar local variables to registers from a large register file physically associated with each SM.

- **Definition:** Per-thread, on-chip storage addressed by instructions without explicit memory loads in the source model.
- **Why it matters:** Register use affects instruction speed, dependency chains, occupancy, and spilling.
- **Used in:** Loop variables, addresses, accumulators, temporary values, and matrix fragments.
- **Interview value:** Registers illustrate the trade-off between per-thread efficiency and the number of resident threads.

2. Core Idea

Registers are like each worker's pockets: values are immediately available, but the workshop has a finite total pocket budget. If each worker needs too many pockets, fewer workers fit in the workshop. If the compiler runs out, some items go to a distant storage area—**register spilling**—which is much slower.

Step by step:

1. Compiler analyzes live variables.
2. It assigns them to per-thread registers where possible.
3. At launch, each block's register demand contributes to SM resource allocation.
4. High demand can reduce resident blocks/warps.
5. Excess live state may spill to thread-local memory backed by device memory and cached through the hierarchy.

3. Important Subtopics

Register allocation and liveness

- **Meaning:** Only simultaneously live values need distinct registers; compiler allocation is architecture-specific.
- **Why:** Long live ranges and unrolling increase pressure.
- **Example:** Keeping many accumulators improves instruction parallelism but consumes registers.
- **Interview angle:** Source variable count does not directly equal register count.

Register pressure

- **Meaning:** Demand for registers relative to availability.
- **Why:** It can reduce occupancy or force spills.
- **Example:** A large per-thread tile uses many accumulators.
- **Interview angle:** Balance occupancy, reuse, and instruction-level parallelism.

Register spilling

- **Meaning:** Values that cannot remain in registers are stored in local memory.
- **Why:** Spills add load/store instructions and memory latency/traffic.
- **Example:** Aggressive loop unrolling creates too many live temporaries.
- **Interview angle:** "Local memory" is private in scope, not necessarily physically on-chip.

Register dependencies

- **Meaning:** An instruction may wait until a previous instruction produces its input register.
- **Why:** Dependency chains limit instruction-level parallelism.

- **Example:** One accumulator updated in a long reduction creates a serial chain; multiple partial accumulators can help.
- **Interview angle:** Fast storage does not remove arithmetic dependency latency.

4. Real-World Example

In matrix multiplication, each thread keeps several output accumulators in registers while tiles are reused from shared memory. More accumulators increase data reuse and instruction-level parallelism, but too many reduce occupancy or spill. Tuned kernels search this trade-off for each architecture.

5. Diagrams / Mental Models

SM register file (finite)

└─ Block A: threads × registers/thread

└─ Block B: threads × registers/thread

└─ Free space insufficient -> another block cannot become resident

register value -> spill store -> local address space -> cache/global-memory path

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Who owns GPU registers?	They are logically private per thread, allocated from an SM register file.	Logical versus physical view.	Saying a block has one register set.
2. Are registers the fastest memory?	They are typically the lowest-latency storage for thread operands, though dependencies and bank/port limits still matter.	Qualified answer.	Calling them unlimited.
3. What is register pressure?	High simultaneous demand for registers, affecting allocation, occupancy, or spills.	Liveness and limits.	Counting source variables only.
4. What is spilling?	Compiler-generated movement of excess values to local memory.	Local memory backing.	Saying values spill into shared memory automatically.
5. How do registers affect occupancy?	More registers per thread/block consume the finite SM pool, possibly reducing resident warps/blocks.	Resource math.	Saying more registers always increase speed.
6. Is local memory on-chip?	Not generally; it is a per-thread address space usually backed by device memory and cached.	Naming trap.	Interpreting "local" as physically near.

Question	Clear answer	Interviewer expects	Common mistake
7. Can threads directly read each other's registers?	Normally no; use shuffle operations, shared memory, or supported group mechanisms.	Scope and exchange.	Dereferencing another thread's variable.
8. How can register pressure be reduced?	Shorten live ranges, reduce unrolling/temporaries or per-thread tile size, and inspect generated code.	Evidence-based tuning.	Forcing an arbitrary cap first.
9. Why can limiting registers hurt?	It may cause spills and extra instructions even if theoretical occupancy rises.	Trade-off.	Optimizing occupancy alone.
10. Are register counts portable?	No; allocation changes with compiler, flags, code, and target architecture.	Compile/device specificity.	Memorizing one count.

7. Deep-Dive Questions

1. **Why are occupancy changes stepwise?** Registers are allocated with hardware granularity per warp/block, so a one-register source change can cross an allocation threshold.
2. **What is register bank conflict?** Some designs partition register storage/ports; certain operand patterns may require extra cycles, though details are architecture-specific.
3. **How can multiple accumulators improve speed?** They break one dependency chain and allow instructions to overlap, at the cost of extra registers.
4. **Why can arrays become local memory?** Dynamic indexing or a size too large for scalarization can prevent the compiler from keeping every element in registers.
5. **Should one always reduce spills?** Usually spills are expensive, but a small spill cost may be acceptable if it enables a better overall schedule; profile the complete kernel.

8. Comparison Tables

Property	Registers	Shared memory	Local memory
Scope	Per thread	Per block	Per thread address space
Physical location	On-chip register file	On-chip scratchpad	Usually device memory, cached
Allocation	Mostly compiler	Programmer/static or dynamic	Compiler/source objects
Cooperation	Via special exchange ops	Direct block-thread sharing	No ordinary cross-thread sharing
Main risk	Pressure lowers occupancy	Capacity/bank conflicts	High-latency traffic/spills

9. Common Mistakes

- Assuming every local variable lives in a register.
- Assuming “local memory” means fast on-chip memory.
- Reducing register count without checking spill traffic.
- Equating occupancy with performance.
- Ignoring compiler target and flags when comparing counts.

10. Edge Cases / Special Cases

- Constants can be folded into instructions and consume no persistent register.
- Addressable arrays, variable indexing, or function calls may alter allocation.
- Debug builds commonly use resources differently and should not represent release performance.
- Register values do not normally persist across kernel launches.
- Architecture-specific matrix APIs may store fragments in registers with opaque mapping.

11. How to Explain in Interview

“Registers hold each thread’s active values and are allocated from a finite SM register file. They are extremely fast, but high per-thread use can reduce the number of resident warps. If allocation cannot keep values in registers, the compiler may spill them to local memory, trading expensive memory traffic for storage.”

12. Quick Revision Notes

- Logical scope: per thread; physical pool: per SM.
- High pressure can reduce occupancy or cause spills.
- Local memory is private by scope, usually off-chip by backing.
- More registers can increase reuse/ILP.
- Inspect compiler reports and profile; do not tune by guesswork.

13. Practice Tasks

1. Compile a kernel with increasing loop unrolling and record registers per thread.
2. Compare occupancy and runtime before and after an artificial register limit.
3. Inspect generated assembly or profiler counters for local load/store spills.
4. Rewrite one long accumulator chain using several partial accumulators and compare results.

14. Final Cheat Sheet

Item	Remember
Core definition	Registers are fast per-thread operands from a finite SM pool.
Why it matters	They trade per-thread efficiency against residency.
Most asked	Register pressure? Spilling? Occupancy effect?
Key comparison	Registers are private and fastest; shared memory is block-shared.
One-line answer	“Registers are cheap to access but expensive in occupancy when each thread uses too many.”

Shared Memory

1. Overview

Shared memory is a programmer-managed, on-chip scratchpad shared by threads in the same block/work-group. CUDA uses "shared memory"; OpenCL often calls the corresponding address space "local memory."

- **Definition:** Low-latency block-scoped storage explicitly loaded and reused by cooperating threads.
- **Why it matters:** It reduces redundant global-memory traffic, enables communication, and helps transform poor access patterns.
- **Used in:** Tiling, reductions, scans, histograms, stencils, transposes, and matrix multiplication.
- **Interview value:** It tests synchronization, bank conflicts, tiling, lifetime, and resource trade-offs.

2. Core Idea

Shared memory is a common worktable inside one SM workshop. Workers fetch material from the distant warehouse once, place it on the table, synchronize, and reuse it many times. The table is fast but small, divided among resident blocks, and organized into banks that can create contention.

Tiled matrix multiplication:

1. Each block owns an output tile.
2. Threads cooperatively load matching **A** and **B** tiles from global memory.
3. A barrier ensures loading is complete.
4. Threads reuse tile values for several multiply-adds.
5. Another barrier protects the table before the next tile overwrites it.

3. Important Subtopics

Scope and lifetime

- **Meaning:** One allocation belongs to a block while it is resident.
- **Why:** Other blocks cannot safely use it, and values disappear at block completion.
- **Example:** Each reduction block has its own partial-sum array.
- **Interview angle:** It is not a device-wide cache or cross-block mailbox.

Tiling and reuse

- **Meaning:** Data is loaded in chunks and reused from shared memory.
- **Why:** Reuse raises arithmetic intensity and effective bandwidth.
- **Example:** A matrix element participates in many products after one global load.
- **Interview angle:** Include halo handling and synchronization.

Bank conflicts

- **Meaning:** Shared memory is split into banks; conflicting lane addresses in one instruction may require serialized service.
- **Why:** Poor layout lowers bandwidth even though storage is on-chip.
- **Example:** Padding a transpose tile from **[32][32]** to **[32][33]** can change bank mapping.

- **Interview angle:** Broadcast of the same address may be handled specially; not every repeated access conflicts.

Capacity and occupancy

- **Meaning:** Per-block shared-memory allocation comes from a finite SM pool.
- **Why:** Large tiles can reduce resident blocks.
- **Example:** Two 48 KiB blocks cannot coexist on an SM with only 64 KiB available for their allocations.
- **Interview angle:** Larger tiles improve reuse but can reduce latency hiding.

4. Real-World Example

A GPU image-convolution block loads a pixel tile plus border halo into shared memory. Neighboring output threads reuse overlapping input pixels. Without tiling, each neighboring thread repeatedly loads the same global pixels. The implementation must handle image boundaries and synchronize after cooperative loading.

5. Diagrams / Mental Models

```

Global memory (large, high latency)
    | coalesced cooperative load
    v
Shared tile owned by Block 7 on SM 2
[data data data halo]
    | barrier
    v
threads reuse nearby values -> compute -> global stores

```

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is shared memory?	Programmer-managed on-chip storage shared by threads in one block.	Scope/location/control.	Calling it global cache.
2. Why is it fast?	It is on-chip and explicitly addressed, avoiding normal global-memory latency when conflict-free.	Qualification.	Claiming one-cycle universally.
3. Who can access it?	Threads belonging to the same block allocation.	Block scope.	Cross-block access.
4. Why use a barrier?	To ensure cooperative writes are complete/visible before reads and to protect phase reuse.	Ordering and visibility.	Barrier after only some threads.
5. What is a bank conflict?	Multiple lane accesses map incompatibly to the same bank,	Banked organization.	Confusing with cache miss.

Question	Clear answer	Interviewer expects	Common mistake
	requiring extra service cycles.		
6. How can conflicts be reduced?	Change indexing/layout, pad arrays, or use suitable access widths after profiling.	Layout transformation.	Padding every array blindly.
7. Static versus dynamic shared memory?	Static size is compile-time declared; dynamic size is supplied at launch and partitioned by the kernel.	Allocation mechanisms.	Thinking dynamic grows during execution.
8. How does it affect occupancy?	Per-block allocation reduces how many blocks can reside on an SM.	Finite pool.	Assuming it is free.
9. Shared memory versus L1?	Shared is explicitly managed and block-addressed; L1 automatically caches eligible accesses. Some GPUs share physical capacity.	Control distinction.	Treating them as identical.
10. When is shared memory unnecessary?	When data has little reuse, caches already work well, or staging overhead exceeds benefit.	Cost-benefit.	Using it in every kernel.

7. Deep-Dive Questions

1. **Why are two barriers often needed per tiled-loop iteration?** One ensures the current load is complete; another ensures all consumers finish before the storage is overwritten for the next tile.
2. **How does shared memory fix a transpose?** Threads load global memory in a coalesced orientation, synchronize, then read the tile transposed; padding can avoid bank conflicts.
3. **What is shared-memory broadcast?** When lanes request the same location, hardware may serve the value efficiently rather than treating it as a harmful conflict.
4. **Can atomics target shared memory?** Yes on common GPUs; they coordinate block-local updates and can be faster than global atomics, though contention still serializes work.
5. **What is double buffering?** Two shared-memory buffers alternate loading and computing, potentially overlapping data movement with arithmetic when supported and carefully synchronized.

8. Comparison Tables

Property	Shared memory	L1 cache	Global memory
Management	Explicit	Automatic	Explicit allocations/accesses
Scope	Block allocation	SM/cache scope, not cooperation semantics	Device allocation
Capacity	Small	Small	Large

Property	Shared memory	L1 cache	Global memory
Lifetime	Block residency	Hardware-managed	Allocation lifetime
Key issue	Banks, barriers, occupancy	Hit rate, locality	Coalescing, latency, bandwidth

9. Common Mistakes

- Reading data before all producer threads finish.
- Placing a block barrier in non-uniform control flow.
- Ignoring bank conflicts.
- Allocating a tile so large that occupancy collapses.
- Staging data with no reuse, adding work without benefit.

10. Edge Cases / Special Cases

- Some architectures combine or configure shared-memory and L1 capacity.
- Bank width/count and conflict behavior are architecture-specific.
- Image and matrix boundaries need padding, conditional loads, or specialized edge blocks.
- A block barrier does not make data available to another block's shared memory.
- Asynchronous copy instructions may require explicit pipeline/barrier semantics beyond a basic barrier.

11. How to Explain in Interview

"Shared memory is a small, explicitly managed on-chip scratchpad shared within a block. Threads cooperatively load reusable data from global memory, synchronize, and reuse it at low latency. Its benefits depend on reuse and conflict-free access, while its finite per-SM capacity can reduce occupancy."

12. Quick Revision Notes

- Block-scoped, on-chip, explicitly managed.
- Key pattern: load → barrier → reuse → barrier before overwrite.
- Improves reuse and can repair global access patterns.
- Watch bank conflicts and capacity-limited occupancy.
- It is not a cross-block communication mechanism.

13. Practice Tasks

1. Implement tiled matrix transpose with and without one padding column.
2. Implement a block reduction using shared memory and barriers.
3. Compare direct and tiled 2D convolution for multiple filter sizes.
4. Deliberately create several strides and measure bank-conflict counters.

14. Final Cheat Sheet

Item	Remember
Core definition	Shared memory is a block-owned on-chip scratchpad.

Item	Remember
Why it matters	It enables reuse, cooperation, and access reordering.
Most asked	Bank conflict? Why barriers? Occupancy effect?
Key comparison	Explicit shared scratchpad versus automatic L1 cache.
One-line answer	"Load once from global, synchronize, and reuse many times within the block."

L1/L2 Cache

1. Overview

GPU caches reduce repeated traffic to device memory. **L1** is small, low-latency, and located close to each SM or SM partition. **L2** is larger, shared across the GPU, and sits between all SMs and global memory.

- **Definition:** Hardware-managed storage holding recently or spatially related memory lines.
- **Why it matters:** Cache behavior changes effective latency, bandwidth demand, atomic behavior, and inter-SM data reuse.
- **Used in:** Nearly every load/store path, depending on instruction, address space, architecture, and cache policy.
- **Interview value:** Interviewers want hierarchy reasoning rather than the claim that every global access always reaches DRAM.

2. Core Idea

Imagine each SM has a small desk drawer (L1), all SMs share a larger supply room (L2), and device DRAM is a distant warehouse. A requested value is cheapest when found nearby. Hardware moves cache-line-sized regions, so nearby accesses help; irregular accesses can fetch much more data than is used.

Typical read path:

1. A warp issues a load.
2. Addresses are coalesced into memory transactions.
3. Eligible requests check L1 according to policy.
4. L1 misses go to shared L2.
5. L2 misses fetch from device memory.
6. Returned lines may populate caches for later accesses.

3. Important Subtopics

Locality

- **Meaning:** Temporal locality reuses data soon; spatial locality accesses nearby addresses.
- **Why:** Both raise cache hit rate and reduce DRAM traffic.
- **Example:** Re-reading a small lookup table versus streaming a huge array once.
- **Interview angle:** Coalescing and caching are distinct: one combines current requests; the other serves reuse.

L1 versus L2 scope

- **Meaning:** L1 is local to an SM/partition; L2 is shared device-wide.
- **Why:** Data produced on one SM may be observed through coherent lower-level mechanisms, while another SM generally cannot use its L1 contents directly.
- **Example:** Separate blocks on different SMs can benefit from L2 reuse.
- **Interview angle:** Cache coherence and visibility rules depend on architecture and the programming memory model.

Cache policies

- **Meaning:** Loads may cache at different levels, bypass a level, stream, or use read-only paths.
- **Why:** Streaming data can evict valuable reusable lines.
- **Example:** Marking one-time data as streaming may preserve a hot working set where supported.
- **Interview angle:** Policies are hints/architecture-specific; correctness must come from synchronization, not expected cache residency.

Working set and thrashing

- **Meaning:** A working set larger than effective cache capacity continually replaces lines.
- **Why:** Nominal cache size does not guarantee hits, especially under many concurrent warps.
- **Example:** Several blocks touch unrelated large tables and compete in L2.
- **Interview angle:** Effective capacity is affected by associativity, partitioning, and contention.

4. Real-World Example

In a graph analytics kernel, neighboring vertices may reuse adjacency metadata. Reordering vertices to improve locality can make blocks reuse L2 lines and reduce random DRAM traffic. However, a graph much larger than cache with random edges remains latency- and bandwidth-limited despite the presence of caches.

5. Diagrams / Mental Models

```
SM 0 -> L1 0 --\
SM 1 -> L1 1 ----> shared L2 slices -> memory controllers -> HBM/GDDR
SM 2 -> L1 2 --/

Coalescing: shapes one warp's requests
Caching:    serves data reused over time/across eligible requesters
```

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. Why do GPUs need caches?	To reduce latency and DRAM traffic for reused or nearby data.	Locality and bandwidth.	Saying GPUs hide all latency without caches.

Question	Clear answer	Interviewer expects	Common mistake
2. L1 versus L2?	L1 is smaller/closer and SM-local; L2 is larger and shared across SMs.	Scope and size/latency relationship.	Saying L2 belongs to each SM.
3. Does every global load use L1?	Not necessarily; behavior depends on architecture, load type, address space, and policy.	Qualified answer.	Universal caching claim.
4. Is a cache hit the same as coalescing?	No. Coalescing combines lanes' requests; a cache hit supplies requested lines from cache.	Clear distinction.	Using terms interchangeably.
5. What is temporal locality?	Reusing the same data within a short time.	Definition/example.	Describing adjacency only.
6. What is spatial locality?	Accessing nearby addresses likely contained in the same/few cache lines.	Cache-line idea.	Confusing with same-time access only.
7. Why can more occupancy hurt cache behavior?	More resident warps may enlarge the active working set and evict useful lines.	Trade-off.	Assuming occupancy has no downside.
8. Can programmers rely on a value staying in cache?	No. Caches are hardware-managed; correctness uses memory-model synchronization.	Correctness separation.	Cache-based synchronization.
9. Why is L2 important for atomics?	Global atomic operations and coherence/serialization commonly interact at or through L2, architecture permitting.	Shared hierarchy point.	Claiming atomics occur in L1 universally.
10. How do you diagnose cache behavior?	Use profiler hit rates, sectors/transactions, DRAM bytes, and controlled access-pattern experiments.	Measurement.	Inferring from runtime alone.

7. Deep-Dive Questions

1. **Why can a high L1 hit rate still be slow?** Hits may serialize due to dependencies, insufficient warps, replayed transactions, or execution bottlenecks; hit-rate percentage also hides access volume.
2. **What is cache-line overfetch?** Hardware transfers a line/sector even if lanes use only a few bytes; scattered accesses waste bandwidth and capacity.
3. **How can loop ordering improve cache use?** It makes reused data accessed close together and keeps the working set within effective cache capacity.
4. **Why is L2 useful across kernels?** Recently used data may remain resident and be reused by a following kernel, but persistence is not guaranteed unless special mechanisms provide controls.

5. **How do memory fences relate to caches?** Fences/order operations enforce memory-model visibility/order within defined scopes; they are not merely cache flush commands and often require matching synchronization/atomics.

8. Comparison Tables

Property	L1	L2	Shared memory
Location/scope	Per SM/partition	Shared across GPU	Per block allocation on SM
Management	Hardware	Hardware	Programmer
Typical size	Smallest cache	Larger cache	Small scratchpad
Main benefit	Lowest cache latency/local reuse	Device-wide reuse, reduce DRAM	Predictable explicit reuse/cooperation
Main uncertainty	Policy/eviction	Contention/eviction	Programmer correctness/banks

9. Common Mistakes

- Assuming all global accesses are uncached.
- Treating cache presence as a reason to ignore coalescing.
- Assuming L1 is shared across SMs.
- Depending on cache residency for correctness.
- Comparing hit rates without comparing bytes and transactions.

10. Edge Cases / Special Cases

- L1 and shared memory may share configurable capacity.
- Stores, atomics, and read-only loads can follow different caching paths.
- Unified-memory page migration operates at a different granularity than cache lines.
- ECC, compression, and sectorized caches affect observed traffic.
- Cache sizes and policies vary by generation; query documentation and profile the target.

11. How to Explain in Interview

"GPU L1 caches are small and close to individual SMs, while L2 is larger and shared by the device. They reduce latency and DRAM traffic when accesses have locality. Coalescing determines how a warp's addresses become transactions; caching determines whether those transactions can be served without DRAM. Correctness must never depend on cache residency."

12. Quick Revision Notes

- L1: close and SM-local; L2: larger and device-shared.
- Locality improves hit rate; overfetch wastes bandwidth.
- Coalescing $\neq$ caching.
- More concurrency can increase cache contention.
- Use profiler bytes/transactions plus hit rates.

13. Practice Tasks

1. Benchmark repeated access to arrays smaller and larger than likely L2 capacity.
2. Compare sequential, strided, and random reads and inspect transactions.
3. Change loop ordering in a stencil or matrix kernel and measure cache metrics.
4. Explain why two kernels may show different time even with the same arithmetic count.

14. Final Cheat Sheet

Item	Remember
Core definition	L1 is near each SM; L2 is shared before DRAM.
Why it matters	Caches reduce latency and external-memory traffic.
Most asked	L1 vs L2? Caching vs coalescing?
Key comparison	Hardware-managed caches versus explicit shared memory.
One-line answer	"Caches reward locality, but access shape and synchronization still matter."

Global Memory

1. Overview

Global memory is the large device-wide memory address space used for kernel inputs, outputs, and persistent data. On a discrete GPU it is normally backed by GDDR or HBM; on an integrated GPU it may share physical system memory.

- **Definition:** Large, high-latency memory accessible by threads across the device and usually by the host through explicit or managed mechanisms.
- **Why it matters:** Global-memory traffic is a dominant performance cost in many kernels.
- **Used in:** Arrays, tensors, frame buffers, graphs, model weights, and inter-kernel data.
- **Interview value:** It tests coalescing, alignment, data layout, transfers, synchronization, and bandwidth reasoning.

2. Core Idea

Global memory is the warehouse serving every workshop. It stores far more than registers or shared memory but takes longer to reach. The goal is not simply to avoid it—inputs and outputs must live somewhere—but to move useful, contiguous chunks and reuse them before fetching again.

For a vector load:

1. Each lane calculates an address.
2. Hardware groups lane requests into memory transactions.
3. Aligned contiguous addresses usually need fewer transactions.
4. Requests travel through caches and possibly device DRAM.
5. While a warp waits, the SM schedules other ready warps.
6. Repeated data should be reused from registers, shared memory, or cache where practical.

3. Important Subtopics

Coalesced access

- **Meaning:** Lane addresses map efficiently to a small number of memory sectors/transactions.
- **Why:** It maximizes useful bytes per transferred byte.
- **Example:** `a[base + lane]` versus `a[base + lane * largeStride]`.
- **Interview angle:** Mention alignment, width, and architecture rather than a simplistic single-transaction guarantee.

Data layout

- **Meaning:** Array of Structures (AoS) interleaves fields; Structure of Arrays (SoA) stores each field contiguously.
- **Why:** If all lanes read one field, SoA often improves coalescing.
- **Example:** `x[N], y[N]` versus `{x,y} points[N]`.
- **Interview angle:** AoS can still be appropriate when each thread consumes entire compact structures; measure the actual pattern.

Host-device movement

- **Meaning:** Discrete devices commonly require transfers over PCIe or a faster interconnect.
- **Why:** Transfer time can dominate short kernels.
- **Example:** Copy once, run several kernels, copy final result rather than round-tripping after every stage.
- **Interview angle:** Pinned memory and asynchronous copies can improve throughput but add constraints/cost.

Latency hiding and bandwidth

- **Meaning:** Many outstanding requests and ready warps hide latency; coalescing and reuse conserve bandwidth.
- **Why:** These solve different problems.
- **Example:** A streaming kernel may hide latency yet still saturate DRAM bandwidth.
- **Interview angle:** More warps cannot exceed physical peak bandwidth indefinitely.

4. Real-World Example

An analytics service scans a large numeric column. Each thread handles adjacent rows, so loads coalesce. The predicate and count require little arithmetic, making the kernel bandwidth-bound. Compressing the column or fusing later operations can reduce bytes moved and improve speed more than adding arithmetic optimization.

5. Diagrams / Mental Models

```

Good warp access:  lane 0 1 2 3 4 ... -> [adjacent words.....]
                   few aligned sectors, most bytes useful

Poor warp access:  lane 0 -> [word]          lane 1 -> [word]          ...
                   many sectors, much transferred data unused
  
```

Registers/shared/cache -> reuse -> fewer global-memory requests

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is global memory?	The large device-accessible address space, usually backed by high-bandwidth device DRAM on discrete GPUs.	Size/scope/backing.	Calling it uncached always.
2. Why is it slow?	DRAM is physically farther and has much higher access latency than on-chip storage, though bandwidth is high.	Latency versus bandwidth.	Saying low bandwidth only.
3. What is coalescing?	Combining a warp's memory addresses into efficient transactions.	Useful bytes/transaction.	Confusing it with cache hits.
4. What access pattern is favorable?	Adjacent lanes accessing aligned adjacent elements of suitable width.	Lane-to-address mapping.	Saying sequential loop order alone.
5. AoS versus SoA for GPUs?	SoA often coalesces field-wise accesses; AoS may be fine when threads use complete compact records.	Pattern-dependent choice.	Claiming SoA always wins.
6. How is global latency hidden?	Keep several ready warps and outstanding operations so other work issues while one request waits.	Concurrency.	Saying latency decreases physically.
7. What is pinned host memory?	Non-pageable host memory that supports efficient DMA and often asynchronous transfer, but is a limited system resource.	Benefit and cost.	Pinning all host memory.
8. What is unified/managed memory?	A unified address model with runtime/hardware placement and migration; locality and page movement still affect performance.	Convenience not magic.	Saying CPU/GPU access costs become equal.
9. How reduce global traffic?	Reuse in registers/shared/cache, fuse kernels, improve layout, compress, or avoid unnecessary intermediates.	Bytes-first optimization.	Only increasing block size.

Question	Clear answer	Interviewer expects	Common mistake
10. Are global writes visible immediately to every thread?	Visibility/order require the programming memory model's synchronization, atomics, and scopes; timing alone is insufficient.	Correct synchronization.	Relying on scheduling order.

7. Deep-Dive Questions

1. **Why can misalignment cost extra transactions?** A request spanning sector/line boundaries must fetch multiple units even if accesses are contiguous.
2. **What is memory-level parallelism?** Multiple independent memory operations remain outstanding concurrently, helping cover long latency.
3. **Why can vectorized loads help?** They may reduce instruction count and improve transaction formation when alignment and layout are suitable; they do not repair fundamentally scattered access.
4. **What is partition camping?** Unfortunate address patterns can overload particular memory partitions/controllers on some designs, reducing available parallel bandwidth.
5. **How do atomics affect global memory?** Atomic updates serialize conflicting operations at a memory location/cache line or through relevant hardware, so contention pattern matters more than the presence of an atomic alone.

8. Comparison Tables

Property	Global memory	Shared memory	Registers
Capacity	Largest	Small	Smallest per thread
Scope	Device allocation	Block	Thread
Latency	Highest	Low	Lowest
Management	Programmer/runtime + caches	Programmer	Compiler
Optimization	Coalesce and reduce traffic	Reuse and avoid banks	Control pressure/dependencies
Lifetime	Allocation-defined	Block lifetime	Thread lifetime

9. Common Mistakes

- Saying global memory has no caches.
- Ignoring alignment and access width.
- Treating unified memory as free transfer.
- Launching many warps to "solve" a saturated bandwidth bottleneck.
- Using inter-block flags without atomics/fences and a valid scheduling design.

10. Edge Cases / Special Cases

- Integrated GPUs may share physical DRAM with CPUs but still face bandwidth contention and coherence costs.
- Zero-copy mapped host memory can avoid explicit copies but often has higher per-access latency; it suits limited access patterns.
- Managed-memory oversubscription can migrate or evict pages and cause severe stalls.
- Error-correcting code and compression can change effective bandwidth/capacity behavior.
- Scatter/gather may be inherently irregular; reordering or batching can help more than shared-memory staging.

11. How to Explain in Interview

“Global memory is the GPU’s large device-wide storage, usually backed by GDDR or HBM. It offers high aggregate bandwidth but high latency. Good kernels make adjacent lanes access aligned adjacent data, keep enough work to hide latency, and reduce total traffic through reuse, fusion, and suitable layouts.”

12. Quick Revision Notes

- Large and device-wide; usually off-chip on discrete GPUs.
- High bandwidth does not mean low latency.
- Coalescing improves transaction efficiency.
- Hide latency with concurrency; reduce bandwidth demand with reuse.
- Managed/unified address space does not eliminate migration cost.

13. Practice Tasks

1. Benchmark sequential, strided, offset, and random reads.
2. Convert a particle update from AoS to SoA and compare transactions/time.
3. Measure kernel-only versus end-to-end time with host transfers.
4. Fuse two simple streaming kernels and calculate bytes eliminated.

14. Final Cheat Sheet

Item	Remember
Core definition	Global memory is large device-wide storage backed mainly by DRAM.
Why it matters	Its traffic limits many GPU kernels.
Most asked	Coalescing? AoS vs SoA? How hide latency?
Key comparison	High capacity/high latency versus tiny fast on-chip storage.
One-line answer	“Coalesce global accesses, hide their latency, and avoid moving bytes twice.”

Constant / Texture Memory

1. Overview

Constant memory is a small read-only device address space optimized for cases where many threads read the same location. **Texture memory** refers to a read-only access path/cache optimized for spatial locality and, through texture objects, optional addressing and filtering features.

- **Definition:** Specialized read-only memory paths serving access patterns that ordinary loads may handle less efficiently.
- **Why it matters:** Correct use can reduce bandwidth and simplify image/sampling operations.
- **Used in:** Kernel parameters, coefficients, lookup tables, images, volume data, and read-only spatial datasets.
- **Interview value:** It tests whether you match memory spaces to access patterns rather than ranking them as universally fast.

2. Core Idea

Constant memory is like a teacher announcing one fact to the entire class: when everyone asks for the same fact, it can be broadcast efficiently. If every student asks for a different page, requests may need separate service.

Texture memory is like a map desk organized by neighborhoods. Nearby map lookups are likely cached together, and the desk can apply rules such as clamping coordinates or interpolating neighboring samples.

Step by step:

1. Host initializes constant data or creates/binds a texture resource.
2. Kernel issues specialized read-only accesses.
3. Constant cache can broadcast a uniform address to a warp efficiently.
4. Texture cache exploits spatial locality in 1D/2D/3D coordinates.
5. Texture hardware may apply addressing conversion or filtering where supported.

3. Important Subtopics

Constant broadcast

- **Meaning:** A warp reading one constant address can receive a broadcast result.
- **Why:** One cached access can serve all lanes.
- **Example:** Every thread reads the same convolution coefficient for a loop iteration.
- **Interview angle:** Different addresses within a warp may serialize or require multiple requests.

Constant capacity and update model

- **Meaning:** Constant space/cache is small and normally populated from the host between dependent kernel uses.
- **Why:** It suits compact, read-mostly uniform data, not large mutable arrays.
- **Example:** Transformation matrix or physical constants.
- **Interview angle:** Device writes are generally not the intended model for constant storage.

Texture locality and addressing

- **Meaning:** Texture reads are optimized for spatial patterns and can support normalized coordinates, clamp/wrap modes, and filtering.

- **Why:** Image algorithms gain both cache behavior and sampling features.
- **Example:** Bilinear sampling during image resize.
- **Interview angle:** Texture cache is not only for graphics, but its exact benefits depend on modern hardware and access pattern.

Read-only data alternatives

- **Meaning:** Ordinary global loads may use L1/L2 or a compiler-selected read-only path.
- **Why:** Modern caches can make explicit texture use unnecessary for simple linear reads.
- **Example:** A contiguous immutable array may perform well through normal cached loads.
- **Interview angle:** Specialized memory is an optimization/feature choice, not a required destination for every read-only value.

4. Real-World Example

An image-resizing kernel stores a small transformation matrix or repeated coefficients in constant memory. Source pixels are accessed through a texture object so hardware handles coordinate clamping and bilinear interpolation while exploiting 2D locality. Output pixels are written to global memory.

5. Diagrams / Mental Models

Constant path:

warp lanes -> same address? -> one cached value broadcast
 -> many addresses? -> multiple/serialized accesses

Texture path:

(x,y) samples -> spatial cache -> optional address mode/filter -> lane result

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is constant memory?	Small device read-only storage/cache path optimized for warp-uniform reads.	Read-only + broadcast.	Calling it per-thread memory.
2. When is constant memory fastest?	When lanes in a warp read the same address and the value is cached.	Uniform access.	"Whenever data is constant."
3. What if lanes read different constant addresses?	The distinct requests may require separate/serialized service, reducing the broadcast benefit.	Address pattern.	Assuming all are parallel.
4. What is texture memory?	A read-only spatially optimized access/cache path, often with sampling/addressing features.	Cache plus functionality.	Saying it is a separate kind of DRAM.

Question	Clear answer	Interviewer expects	Common mistake
5. Is texture memory only for images?	No; it can serve other read-only spatial/locality patterns, though image sampling is its clearest use.	Generality with fit.	Restricting it to graphics.
6. Texture versus global memory?	The underlying storage may be device memory; texture uses a specialized access path/cache and optional sampling semantics.	Path versus backing.	Treating them as physically unrelated.
7. Can texture hardware interpolate?	With suitable texture formats/configuration it can perform filtering such as linear interpolation, subject to precision/format rules.	Hardware sampling.	Assuming every texture load filters.
8. Can kernels write constant memory?	Constant address space is intended as device read-only; host/runtime updates it outside dependent execution.	Mutability model.	Using it as shared output.
9. Why not put every read-only array in constant memory?	Capacity is limited and divergent addresses lose broadcast efficiency; regular caches may be better.	Pattern/capacity.	"Constant is always fastest."
10. How choose between constant and texture?	Uniform small values favor constant; spatial/coordinated sampling favors texture; linear cached reads may need neither.	Decision by pattern/features.	Choosing by data type alone.

7. Deep-Dive Questions

- 1. Why do constant accesses serialize for different addresses?** Broadcast hardware efficiently serves uniform requests; multiple unique words require multiple transactions, even though exact scheduling is architecture-specific.
- 2. What numerical caveat comes with texture filtering?** Sampling and interpolation precision/rounding may differ from explicit full-precision arithmetic; validate accuracy requirements.
- 3. How do texture caches handle 2D locality?** Their cache organization/request path is designed to retain nearby spatial samples better than purely linear locality in relevant workloads.
- 4. Can a texture resource view reinterpret data?** APIs may allow resource/view formats and coordinate conventions, but conversions and legality depend on the binding and hardware.
- 5. Why are normal read-only loads often competitive now?** Modern GPUs have capable L1/L2/read-only caching, so explicit texture paths are most compelling for sampling features or proven spatial-cache benefits.

8. Comparison Tables

Property	Constant memory	Texture path	Ordinary global load
Device access	Read-only	Read-only sampling	Read/write according to allocation
Best pattern	Same address across lanes	Spatial locality/coordinate sampling	Coalesced linear/general access
Special feature	Warp broadcast	Address modes/filtering/conversion	Generality
Capacity	Small specialized space/cache	Cache backed by resource memory	Large global allocation
Main penalty	Distinct lane addresses	Setup/format constraints, poor locality	Latency/traffic if not coalesced/reused

9. Common Mistakes

- Assuming “read-only” automatically means constant memory.
- Using constant memory for lane-random table lookups.
- Believing texture memory is physically separate DRAM.
- Expecting filtering without configuring it.
- Using specialized paths without measuring against modern normal loads.

10. Edge Cases / Special Cases

- Constant parameters may be passed through special parameter storage with implementation-specific limits.
- A uniform address that misses cache still benefits differently from a hit; broadcast does not eliminate all latency.
- Texture boundary modes change correctness at edges, not only performance.
- Normalized coordinates and filtering introduce coordinate and precision conventions.
- Exact constant sizes, cache behavior, texture formats, and APIs vary by device generation.

11. How to Explain in Interview

“Constant memory is a small read-only path that is especially efficient when an entire warp reads the same address because the value can be broadcast. Texture memory is a read-only spatial access path that can cache nearby coordinates and provide addressing or filtering. I choose them by access pattern and required features, not because either is universally faster than global loads.”

12. Quick Revision Notes

- Constant: small, read-only, best for warp-uniform addresses.
- Texture: read-only, spatial locality, optional sampling features.
- Different constant addresses reduce broadcast efficiency.
- Texture/global can share underlying device storage; the access path differs.
- Modern normal caches may already be sufficient.

13. Practice Tasks

1. Store a small coefficient table in global and constant memory; test uniform and lane-random indices.
2. Implement nearest and bilinear image sampling manually, then compare a texture-object version.
3. Test boundary modes on coordinates outside an image.
4. Profile spatial, linear, and random read patterns through available read-only paths.

14. Final Cheat Sheet

Item	Remember
Core definition	Constant broadcasts uniform reads; texture optimizes spatial sampling.
Why it matters	Specialized patterns reduce traffic or add hardware sampling.
Most asked	When is constant fast? Texture vs global?
Key comparison	Uniform broadcast versus spatially cached sampling.
One-line answer	"Use constant for shared facts and texture for spatial samples."

Memory Bandwidth vs Compute Throughput

1. Overview

Memory bandwidth is the rate at which bytes can move through a memory path, commonly measured in GB/s or TB/s. **Compute throughput** is the rate at which arithmetic operations can execute, commonly measured in FLOP/s or operations/s. Kernel speed is limited by whichever required resource cannot supply work fast enough.

- **Definition:** Bandwidth measures data movement capacity; compute throughput measures arithmetic capacity.
- **Why it matters:** It tells you whether to optimize bytes moved or operations/instruction efficiency.
- **Used in:** Performance modeling, kernel design, profiling, hardware selection, and capacity planning.
- **Interview value:** The roofline model and arithmetic intensity reveal whether a candidate can reason quantitatively rather than repeat optimization rules.

2. Core Idea

Imagine a factory with a delivery conveyor and processing machines. If materials arrive slowly, adding machines does nothing: the job is **memory-bound**. If material piles up while machines work, the job is **compute-bound**.

Define **arithmetic intensity**:

$AI = \text{useful arithmetic operations} / \text{bytes transferred from the measured memory level}$

Example vector addition $C = A + B$ with 32-bit floats:

1. Read A : 4 bytes.

2. Read **B**: 4 bytes.
3. Write **C**: 4 bytes (ignoring additional transaction effects for the simple model).
4. Perform 1 floating-point add.
5. **AI** $\approx 1/12$ **FLOP/byte**, so it is normally bandwidth-bound.

For a roofline estimate:

$\text{attainable performance} \leq \min(\text{peak compute}, \text{memory bandwidth} \times \text{AI})$

3. Important Subtopics

Arithmetic intensity

- **Meaning:** Work performed per byte transferred at a specified hierarchy level.
- **Why:** It places a kernel on the memory-bound/compute-bound spectrum.
- **Example:** Matrix multiplication reuses tiles and has much higher intensity than vector addition.
- **Interview angle:** State which bytes and operations are counted; cache reuse changes DRAM-level intensity.

Roofline model

- **Meaning:** A performance upper bound formed by a sloped bandwidth roof and a flat compute roof.
- **Why:** It identifies the likely limiting resource and plausible maximum.
- **Example:** With 1 TB/s and AI 10 FLOP/B, bandwidth permits 10 TFLOP/s; a 60-TFLOP/s GPU remains memory-bound at that intensity.
- **Interview angle:** It is a bound/model, not a guarantee.

Effective versus peak rates

- **Meaning:** Datasheet peaks assume favorable instruction mix, precision, clocks, and access patterns; effective rates reflect the workload.
- **Why:** Poor coalescing, dependencies, insufficient parallelism, or throttling lower achieved performance.
- **Example:** Scattered reads achieve a fraction of peak DRAM bandwidth.
- **Interview angle:** Compare achieved bytes/FLOPs with relevant theoretical ceilings.

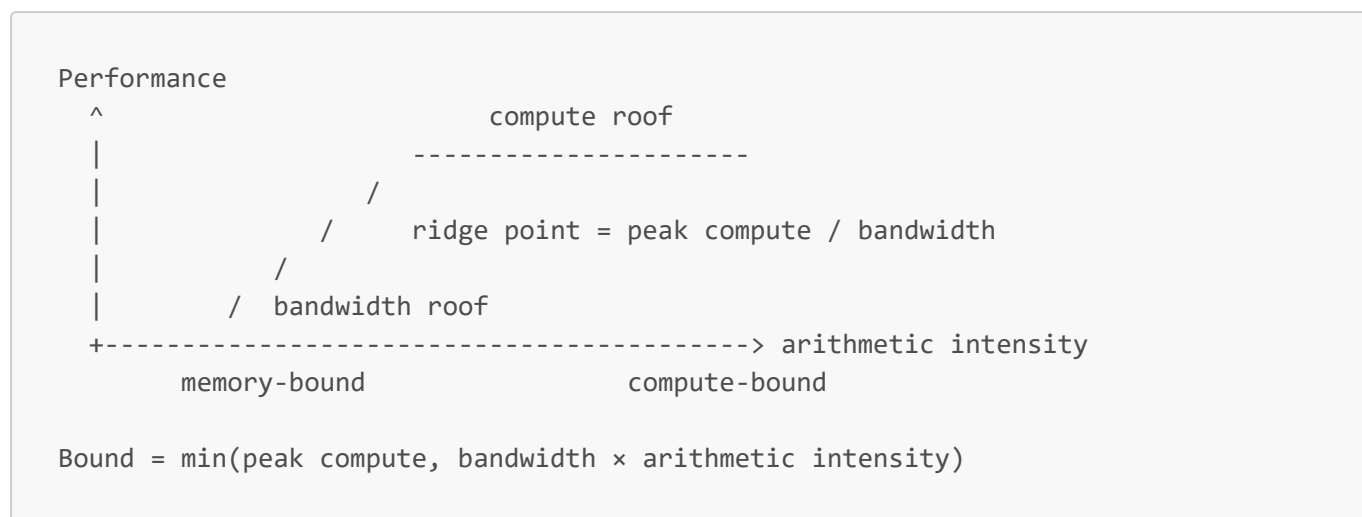
Moving the bottleneck

- **Meaning:** Optimization can shift a kernel from memory-bound to compute-bound or to another limit.
- **Why:** After each meaningful change, re-profile.
- **Example:** Fusion eliminates intermediate traffic, after which special-function throughput becomes limiting.
- **Interview angle:** Do not continue bandwidth optimizations after a different roof dominates.

4. Real-World Example

A neural-network elementwise activation reads and writes each tensor element but performs little math, so it is commonly memory-bandwidth-bound. Fusing it into a preceding matrix operation avoids a separate read/write round trip. The matrix operation itself may be compute-bound on Tensor Cores if tiles are reused efficiently, or bandwidth-bound if the matrix is small or weights are streamed with little reuse.

5. Diagrams / Mental Models



6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. What is memory bandwidth?	Bytes transferable per second through a specified memory level/path.	Units and level.	Confusing with latency.
2. What is compute throughput?	Arithmetic operations executable per second for a specified type/instruction class.	Precision/instruction qualification.	Quoting one FLOPS number universally.
3. What is arithmetic intensity?	Operations performed per byte transferred at a chosen memory level.	FLOP/byte and scope.	Operations per element only.
4. What is a memory-bound kernel?	Performance is limited mainly by data-movement bandwidth/latency rather than arithmetic units.	Evidence and implication.	"It accesses memory."
5. What is a compute-bound kernel?	Execution-unit capacity or dependencies dominate after data is supplied sufficiently.	Compute roof.	"It has many operations."
6. State the roofline bound.	$P \leq \min(P_{\text{peak}}, BW \times AI)$.	Formula and meaning.	Adding the ceilings.
7. How optimize memory-bound code?	Move fewer bytes, coalesce, reuse/cache, fuse stages, use compact types, and overlap transfers where relevant.	Reduce traffic first.	Micro-optimizing arithmetic.
8. How optimize compute-bound code?	Improve instruction mix, use specialized units/precision, reduce dependencies/divergence, and increase unit utilization.	Match execution resource.	Only increasing memory bandwidth.

Question	Clear answer	Interviewer expects	Common mistake
9. Is high occupancy proof of high performance?	No. It indicates resident warps, not achieved bandwidth, lane activity, or compute issue.	Metric distinction.	Treating occupancy as throughput.
10. How determine the bottleneck?	Estimate AI/roofline, then confirm with profiler utilization, bytes, stalls, and controlled experiments.	Model plus measurement.	Guessing from source alone.

7. Deep-Dive Questions

1. **Calculate the ridge point.** It is $\text{peak compute} / \text{peak bandwidth}$ in FLOP/byte. Kernels below it are bandwidth-limited in the simple roofline model; above it can be compute-limited.
2. **Why does cache-level roofline analysis matter?** A kernel can be DRAM-efficient yet limited by L1/L2 bandwidth or shared-memory/issue throughput; each hierarchy level has its own byte traffic and roof.
3. **How can reducing precision help both sides?** Smaller types move fewer bytes and may unlock higher arithmetic throughput or Tensor Cores, but require accuracy/range validation.
4. **Why can adding arithmetic make a memory-bound kernel faster?** Recomputing a cheap value can avoid loading it, reducing scarce bandwidth enough to outweigh extra compute.
5. **How does kernel fusion alter intensity?** It keeps intermediates in registers/shared/cache instead of writing and rereading global memory, reducing bytes per useful operation; register pressure and lost parallelism can limit the gain.

8. Comparison Tables

Dimension	Memory-bound	Compute-bound
Main limit	Data delivery/traffic	Execution-unit capacity/dependencies
Typical arithmetic intensity	Low	High
Profiler clue	High memory throughput, low compute utilization	High relevant-unit utilization
Best first optimization	Reduce/coalesce/reuse bytes	Improve math mapping/instruction efficiency
Hardware upgrade	More effective bandwidth/cache	More relevant compute throughput
Example	Vector add, simple scan	Well-tiled large GEMM
Quantity	Formula/unit	Caution
Bandwidth	bytes / second	Specify DRAM, L2, L1, etc.
Throughput	operations / second	Specify type and what counts as an operation.

Quantity	Formula/unit	Caution
Arithmetic intensity	operations / byte	Count actual relevant-level traffic when possible.
Ridge point	peak throughput / peak bandwidth	Peak values may not be simultaneously sustainable.

9. Common Mistakes

- Confusing memory latency with memory bandwidth.
- Labeling code memory-bound only because it reads memory.
- Counting requested bytes while ignoring inefficient transactions or writes.
- Comparing achieved FP32 work against a tensor/low-precision peak.
- Treating the roofline prediction as guaranteed performance.

10. Edge Cases / Special Cases

- Some kernels are latency-bound rather than bandwidth-saturated because accesses are dependent or insufficiently parallel.
- Integer, special-function, load/store, Tensor Core, and instruction-issue ceilings differ from headline FP throughput.
- Thermal/power throttling and clock variability lower sustainable roofs.
- Data compression may raise effective bandwidth when hardware transfers fewer physical bytes.
- Atomics, synchronization, divergence, and launch overhead can dominate outside the simple two-roof model.

11. How to Explain in Interview

“Memory bandwidth is how many bytes per second the GPU can deliver; compute throughput is how many operations per second it can execute. Arithmetic intensity connects them. The roofline estimate is the minimum of peak compute and bandwidth times operations per byte. Low-intensity kernels should reduce data traffic; high-intensity kernels should improve execution-unit utilization.”

12. Quick Revision Notes

- Bandwidth: bytes/s; throughput: operations/s; latency: time per access.
- $AI = \text{operations} / \text{bytes}$.
- $P \leq \min(P_{\text{peak}}, BW \times AI)$.
- Ridge point: P_{peak}/BW .
- Model first, profile second, optimize the measured bottleneck.

13. Practice Tasks

1. Compute arithmetic intensity for vector add, SAXPY, reduction, and naive/tiled matrix multiplication.
2. Given 60 TFLOP/s and 1.5 TB/s, calculate the 40 FLOP/B ridge point and classify sample kernels.
3. Measure effective bandwidth as $\text{useful bytes} / \text{kernel time}$ for a copy kernel.
4. Fuse two elementwise kernels and compare bytes, register use, occupancy, and time.
5. Build a simple roofline plot from profiler-measured FLOPs and DRAM bytes.

14. Final Cheat Sheet

Item	Remember
Core definition	Bandwidth moves bytes; compute throughput performs operations.
Why it matters	The tighter roof determines what optimization can help.
Most asked	Arithmetic intensity? Roofline? Memory- vs compute-bound?
Key comparison	Low AI favors bandwidth limit; high AI can reach compute limit.
One-line answer	"Performance cannot exceed the smaller of the compute roof and bandwidth times arithmetic intensity."

Whole-Guide Interview Checklist

Before answering any GPU architecture question, identify:

1. **Work hierarchy:** thread, warp/wavefront, block, grid, SM.
2. **Storage hierarchy:** register, shared/L1, L2, global memory, specialized read-only paths.
3. **Access pattern:** uniform, contiguous/coalesced, reused, strided, or random.
4. **Control pattern:** uniform or divergent within a warp/wavefront.
5. **Resource limit:** registers, shared memory, resident warps, bandwidth, compute units, or synchronization.
6. **End-to-end cost:** launch, transfers/migration, kernel execution, and synchronization.
7. **Evidence:** a quantitative model followed by profiler measurements on the target GPU.